Lab 2: Full Path Tracer

Advanced Computer Graphics, 2026–2027
Stuart Lance Richards Weir (NIA 288001) · Gerard Borràs (NIA 266738)

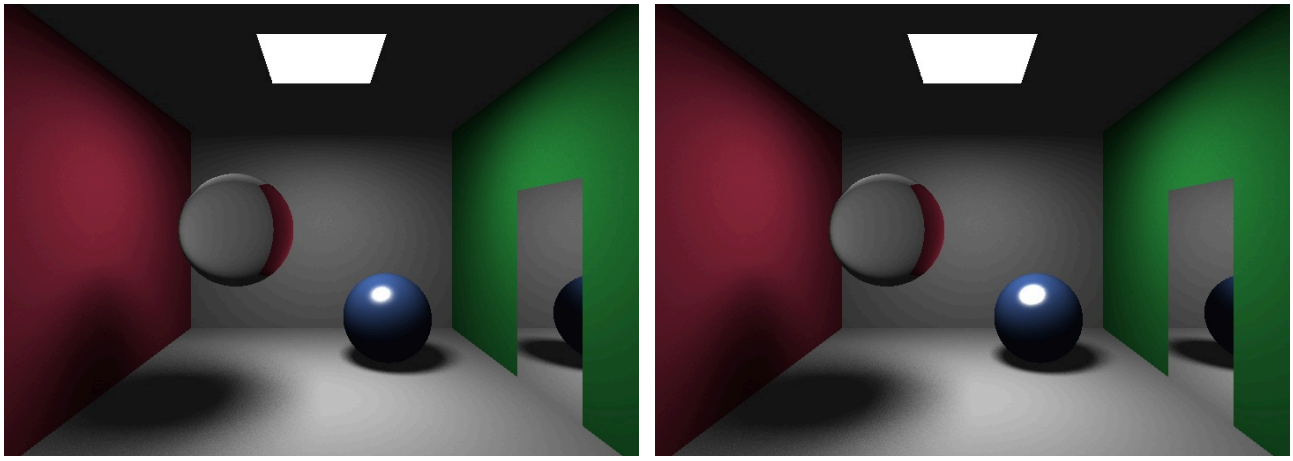
1. Overview

We completed all six tasks: the energy-conserving Phong BRDF, two direct-lighting integrators (hemisphere sampling and light-area sampling), a pure path tracer, and a Next Event Estimation (NEE) integrator that also handles mirrors and glass. Each integrator is its own `Shader` subclass. To turn one random estimate per ray into an image we added the samples-per-pixel loop of the Annex: each pixel averages `spp` independent estimates. All renders are our own, at 720×512, using the new Cornell box with an area light ($L_e = 25$, 2×2 m) on the ceiling. Single-threaded in a Release build, 256 spp take about 18 s for the direct integrators, 100 s for the path tracer and 125 s for NEE.

Comparing renders. Noise makes it hard to judge by eye whether two integrators give the same result, so where we quote a percentage we compare numbers instead. We average the pixel values over the same region of both images (for example everything below the light, or a box around one object) and report the relative difference of the two averages. Where noisy pixels can be brighter than the image format allows, we average the renderer's floating-point values before they are clipped, since clipping removes more brightness from the noisier image.

2. Energy-conserving Phong (Task 4.1)

In Lab 1 the specular lobe $k_s(\boldsymbol{\omega}_r \cdot \boldsymbol{\omega}_o)^\alpha$ was not normalised: as α grows the lobe gets narrower, so the total energy it reflects falls even though k_s stays the same. Integrated over the hemisphere, $\cos^\alpha$ times the cosine factor gives about $2\pi/(\alpha+2)$, so multiplying by $(\alpha+2)/(2\pi)$ makes the lobe reflect a fraction k_s of the incoming energy for every α. With $k_d + k_s \le 1$ the material therefore never reflects more than it receives. For the $\alpha = 20$ blue sphere the factor is $22/2\pi \approx 3.5$, so its highlight grows (Figure 1): in a box around the sphere, the number of fully white pixels roughly doubles (223 ⟶ 468) and the average brightness rises by about 10%. A sharper lobe (larger α) gets a larger factor, because it concentrates the same energy in a smaller cone.



Lab 1 Phong — energy-conserving Phong

Figure 1: Task 4.1: area direct lighting with the Lab 1 Phong (left) and the normalised Phong (right).

3. Direct illumination (Tasks 4.2.1 and 4.2.2)

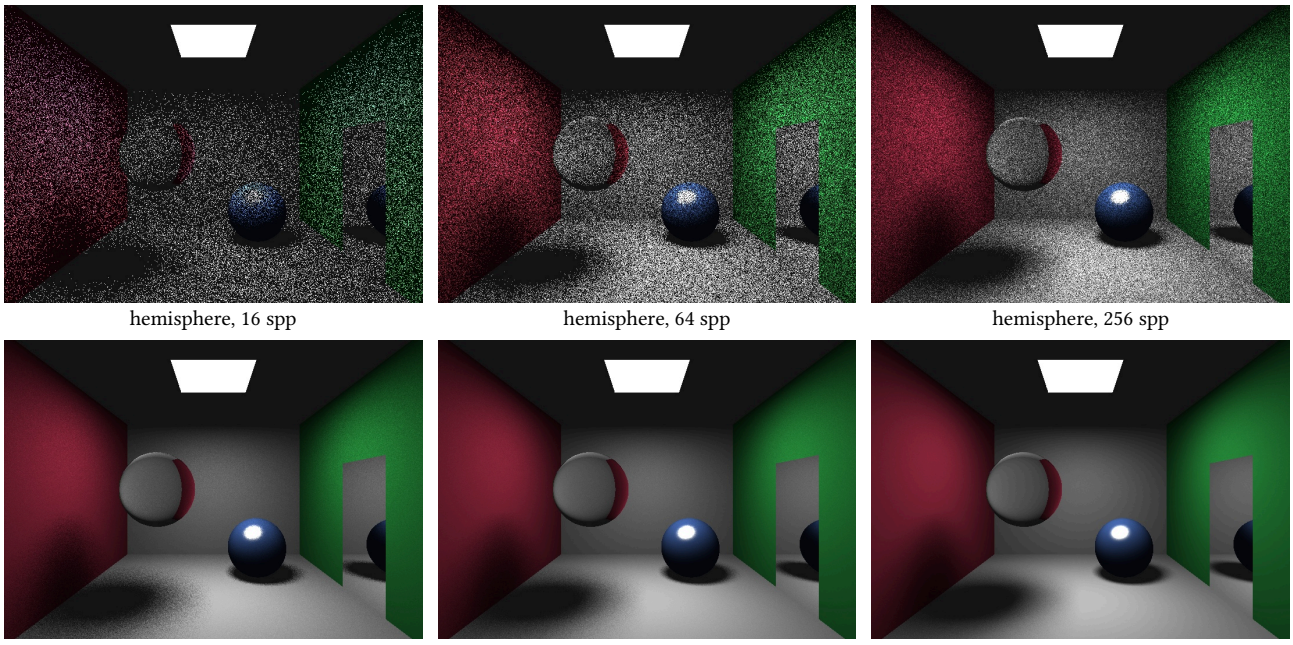
Both integrators shade a Phong point as $I_a k_d + L_o^{\text{dir}}$, keeping the constant ambient term ($I_a = 0.1$) as a stand-in for indirect light. Rays that hit the light return its L_e (only from the emitting side, where $\boldsymbol{n}_y \cdot -\boldsymbol{\omega}_i > 0$), and mirrors and glass recurse exactly as in Lab 1.

Hemisphere sampling (4.2.1). We draw one uniform direction $\boldsymbol{\omega}_i$ around the normal ($p = 1/2\pi$), trace it, and if the first thing it hits is the light we add $L_e f_r(\boldsymbol{\omega}_i \cdot \boldsymbol{n})/p$; otherwise the sample contributes nothing. The estimate is correct but very noisy (Figure 2, top): seen from the floor the light covers a tiny part of the hemisphere, so almost every sample returns 0 and a few return a very large value.

Area sampling (4.2.2). Here we sample the light instead of the hemisphere. A uniform point on the rectangle is $y = \text{corner} + u\boldsymbol{v}_1 + v\boldsymbol{v}_2$ with u, v uniform in $[0,1]$, so its density is $1/A$ with $A = |\boldsymbol{v}_1 \times \boldsymbol{v}_2|$ (the brief calls this function `generateRandomPoint()`; the framework's interface names it `sampleLightPosition()`). Changing the integral from solid angle to area brings in the geometric term $G = \max(0, \boldsymbol{n}_x \cdot \boldsymbol{\omega}_i)\max\big(0, -\boldsymbol{\omega}_i \cdot \boldsymbol{n}_y\big)/\ |y-x|^2$: the cosine at each end measures how much each surface faces the other, and $1/r^2$ how much smaller the light looks from further away. A

shadow ray then checks the visibility $V(x, y)$. Every sample now aims at the light, so it contributes unless it is blocked, and the soft shadow appears almost immediately: area sampling at 16 spp is already smoother than hemisphere sampling at 256 (Figure 2). Both estimators converge to the same image: averaged over everything below the light, the two 256 spp renders differ by under 1%.

One detail of this scene matters here: the light lies exactly in the ceiling plane, so a shadow ray that ends at y can also hit the ceiling at the same distance, and rounding decides whether the light counts as blocked. When we tried it, the image came out speckled and, averaged over the same region, 39% darker. We end the shadow ray at $|y - x| - \varepsilon$ instead.



hemisphere, 16 spp | hemisphere, 64 spp | hemisphere, 256 spp

area, 16 spp | area, 64 spp | area, 256 spp

Figure 2: Tasks 4.2.1 (top) and 4.2.2 (bottom) at 16, 64 and 256 spp.

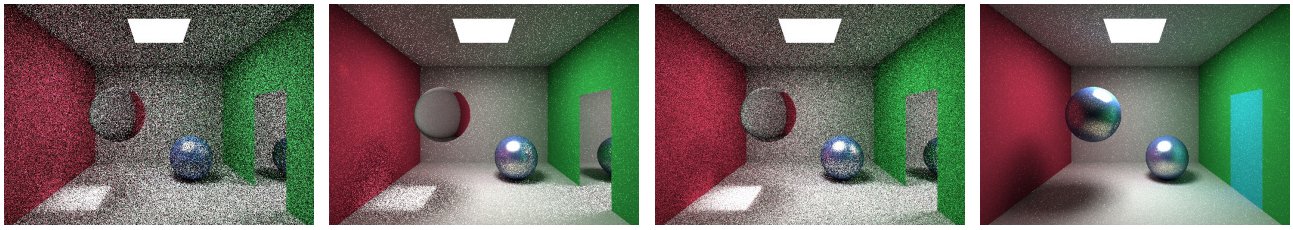
4. Pure path tracing (Task 4.3.1)

The path tracer (Algorithm 1) drops the ambient term and estimates the full rendering equation. At each hit it adds the emitted light L_e and continues the path in one random hemisphere direction, weighting what comes back by $f_r \cos\theta / p$, until `maxDepth` (5) bounces. Mirrors and glass have a delta BRDF, so the path simply continues along the reflected or refracted direction with weight 1. The image (Figure 3) now shows effects the ambient term could not: the ceiling is lit by light bouncing off the floor, the walls tint nearby surfaces red and green (colour bleeding), and a **caustic** appears on the floor where the glass sphere focuses the light. The price is noise: a path only gets light if one of its random bounces happens to hit the small light, which is the same problem as 4.2.1 at every bounce.

5. Next Event Estimation (Tasks 4.3.2 and 4.3.3)

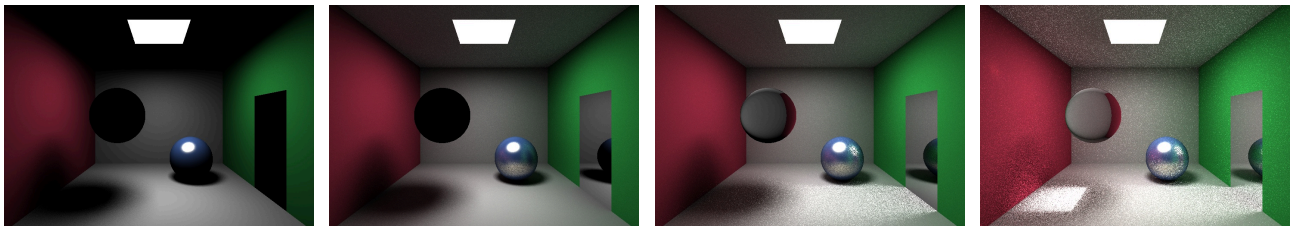
NEE (Algorithm 2) splits the reflected light into direct and indirect parts. At every diffuse or glossy point it adds the direct light with area sampling (the same estimator as 4.2.2, which we reuse) and then continues with one random bounce for the indirect light. Because the direct part already accounts for the light, the emission of a light reached by a random bounce must **not** be added again: that would count it twice. So L_e is only added at the first hit. With every path vertex now connected to the light, NEE converges much faster than pure path tracing (Figure 3, same 64 spp). To check it estimates the same thing, we averaged the unclipped values over five regions (both spheres, floor, back wall and ceiling) in both renders of the Phong-only scene: they agree to within 3%. In the Phong-only scene the spheres look silvery rather than blue because their specular coefficient is larger than their diffuse one and, with the energy-conserving lobe, they mostly reflect the bright room.

Path length. Every bounce, diffuse or specular, counts towards `maxDepth` (Figure 4). With depth 1 NEE gives direct light only: the ceiling, which the light cannot see, is black, and so are the mirror and the glass sphere. Depth 2 lights the ceiling and adds most of the colour bleeding, and the mirror works, but the glass is still black because seeing through it takes two refractions. At depth 3 the glass appears; the caustic needs four bounces (floor, into the glass, out of it, then the light), so it only shows up at depth 5. The diffuse lighting itself barely changes after three bounces, because every bounce multiplies the light by an albedo below 1.



path tracing, 64 spp | NEE, 64 spp | path tracing, 256 spp | NEE, Phong only, 256 spp

Figure 3: Tasks 4.3.1–4.3.2: pure path tracing and NEE at the same 64 spp, pure path tracing at 256 spp, and NEE in the Phong-only scene.

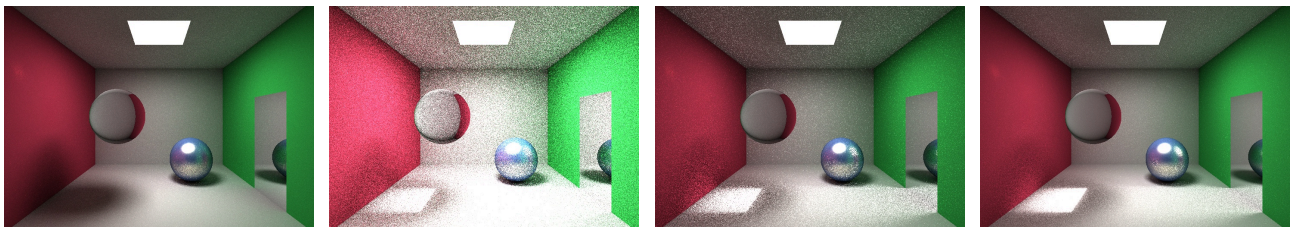


depth 1 | depth 2 | depth 3 | depth 5

Figure 4: NEE with maximum path depth 1, 2, 3 and 5 (256 spp).

Mirrors and glass (4.3.3). The rule "never add L_e after a bounce" breaks for specular surfaces. Direct light sampling cannot find a light *through* a mirror or glass: a shadow ray from the floor towards the light is blocked by the sphere, and a mirror only reflects light from one exact direction, which a random point on the light will never hit. That light can only be found by following the specular path until it reaches the light, so it must be counted when it does. We store this decision on the ray, as the brief suggests: a `countEmission` flag that is true for camera rays and for rays leaving a mirror or glass, and false for random bounces from a diffuse or glossy surface. When a ray hits a light we add L_e only if its flag is set.

Figure 5 shows why both halves of the rule matter. If L_e is never counted after a specular bounce, the caustic disappears, because the path floor $\rightarrow$ glass $\rightarrow$ light is the only way it is found. If L_e is counted after every bounce, light reaching diffuse surfaces is counted twice (once by light sampling, once by the bounce) and the whole image is too bright. Our final render uses 1024 spp.



never after specular | after every bounce | our flag | final, 1024 spp

Figure 5: Task 4.3.3: NEE never counting L_e after mirror/glass, counting it after every bounce (double counting), our flag (256 spp), and the final render at 1024 spp.